Real-world — a FastAPI API + a form page — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

- Hint 1 — a typed endpoint
- Hint 2 — add a form, and test with TestClient

Hint 1 — a typed endpoint

The JSON endpoint is a Pydantic request model plus a function — the validation is automatic:

```
class ClassifyRequest(BaseModel):
    text: str

@app.post("/classify")
def classify_route(body: ClassifyRequest):
    return {"label": classify(body.text)}
```

A request missing `text` is rejected with a `422` before your code runs — you never wrote that check. Open `/docs` and FastAPI has already built interactive docs from the types. This is exactly Doc12's FastAPI.

Hint 2 — add a form, and test with TestClient

FastAPI can serve HTML too, so a human gets a form:

```
from fastapi.responses import HTMLResponse
@app.get("/", response_class=HTMLResponse)
def form():
    return "<form method='post' action='/classify'><input name='text'></form>"
```

Test both the API and the form in-process, no server:

```
from fastapi.testclient import TestClient
client = TestClient(app)
client.post("/classify", json={"text":"amazing"}).json()   # label: positive
client.post("/classify", json={}).status_code              # 422 (validation)
```

For a real LLM pipeline, make the endpoint `async def` and `await` the call, so one worker handles many slow requests. The full code is in the Solution.